



Análisis y Desarrollo de Software

Evidencia 2: Desarrollo del Frontend Proyecto Memorize Evolutivo

Interfaces responsivas del juego web multijugador de memoria

Jose David Sandoval Caceres
Kevin Gregorio Mora

Ficha 3172526

SERVICIO NACIONAL DE APRENDIZAJE SENA
Atención al sector agropecuario
Piedecuesta - Santander

Este documento presenta la evidencia del desarrollo del frontend: el enlace al repositorio publico donde se aloja el codigo, los fragmentos de codigo mas relevantes (estructura de componentes principales y conexion con la API) y las capturas de pantalla de las interfaces ya renderizadas en el navegador.

1. Objetivo y alcance del frontend

El frontend es la parte de la aplicacion con la que interactua el usuario. En este proyecto se tradujo el prototipo a codigo con **React 18 + TypeScript + Vite**, con estilos en **Tailwind CSS**, animaciones con **Framer Motion**, iconos **Lucide** y comunicacion en tiempo real con **Socket.IO Client**. La interfaz es responsiva, usa componentes reutilizables y se comunica correctamente con los servicios del servidor (API REST + WebSocket).

Capa	Tecnologia / version
Framework UI	React 18.2 + TypeScript 5.9
Build tool / dev server	Vite 7.2 (puerto 5173)
Estilos	Tailwind CSS 3.4
Animaciones	Framer Motion 10.18
Tiempo real	Socket.IO Client 4.7
Componentes base	Radix UI + Lucide React

Estructura de carpetas del frontend dentro del repositorio:

frontend/src (estructura real del repositorio)

```
frontend/src/  
App.tsx          # Enrutador por estado + conexion API/Socket  
main.tsx         # Punto de entrada React  
components/      # 35 componentes (pantallas y juego)  
  LoginScreen.tsx, RegisterScreen.tsx, LobbyScreen.tsx, ...  
  GameScreen.tsx, InfiniteMode.tsx, ChallengeMode.tsx, ...  
  BossFight.tsx, BossSelect.tsx, TiendaScreen.tsx, ...  
  RankedScreen.tsx, ProfileScreen.tsx, AdminPanel.tsx, ...  
lib/             # Logica transversal  
  socket.ts      # Singleton Socket.IO  
  playerEvolution.ts # 10 rangos / sistema XP  
  shopSystem.ts, evolutionSystem.ts  
styles/         # Estilos globales + Tailwind
```

2. Enlace al repositorio

El codigo del frontend se aloja en el repositorio publico de GitHub del proyecto, carpeta **frontend/**, rama **main**:

<https://github.com/kevinmora18/memorize>

El repositorio es publico y contiene tanto el frontend (React + Vite) como el backend (Express + Socket.IO), la configuracion de despliegue (**render.yaml**, Dockerfiles) y la documentacion del proyecto. Ultimo commit verificado: arquitectura POO + SOLID completa.

3. Estructura del componente principal (App.tsx)

La navegacion se gestiona por estado (**currentScreen**) sin libreria de rutas: cada pantalla es un componente React que se monta segun el estado, con transiciones por **LoadingScreen**. Al montar, la app abre el canal Socket.IO (**rooms:update**, **room:started**) y hace un fetch inicial de salas como respaldo si

el WebSocket aun no entrega datos:

frontend/src/App.tsx - socket + carga inicial de salas (extracto real)

```
import socket from './lib/socket';

export type GameScreen = 'login' | 'register' | 'lobby' | ... | 'tienda' | 'admin';

useEffect(() => {
  socket.on('rooms:update', (data: Room[]) => setRooms(data));
  socket.on('player:joined', (payload: any) => {...});
  socket.on('room:started', (payload: any) => {
    setCurrentScreen('multiplayerGame');
  });
  (async () => {
    try {
      const res = await fetch(`${API_BASE}/api/rooms`);
      if (res.ok) setRooms(await res.json());
    } catch (err) { console.error(...); }
  })();
  return () => { socket.off(...); };
}, []);
```

Autenticacion contra la API (POST /api/auth/login):

El login invoca al endpoint de autenticacion del backend y construye el jugador con los datos reales (rol, nivel, XP, monedas, gemas). Si el servidor no responde, aplica un respaldo local para no bloquear la experiencia:

frontend/src/App.tsx - handleLoginSuccess (extracto real)

```
const res = await fetch(`${API_BASE}/api/auth/login`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email }),
});
if (res.ok) {
  const userData = await res.json();
  const player: Player = {
    id: userData.id, email: userData.email, teamId: 0,
    role: userData.role, level: userData.level,
    xp: userData.xp, coins: userData.coins, gems: userData.gems
  };
  setCurrentUser(player);
  setPostLoadingScreen('lobby');
  setCurrentScreen('loading');
} else { /* respaldo local: jugador temporal y continuar al lobby */ }
```

4. Conexion con la API y tiempo real

Singleton de Socket.IO:

frontend/src/lib/socket.ts (archivo completo)

```
import { io } from 'socket.io-client';

const API_BASE = import.meta.env.VITE_API_BASE || 'http://localhost:5175';
const socket = io(API_BASE, { autoConnect: true });

export default socket;
```

La URL base se configura por entorno (**VITE_API_BASE**, ver **frontend/.env** y **.env.production**), de modo que en desarrollo apunta a **http://localhost:5175** y en produccion al backend desplegado en Render sin cambiar codigo.

Llamadas REST desde los componentes:

frontend/src/components/LobbyScreen.tsx - carga de moneda del usuario (extracto real)

```
useEffect(() => {
  const stats = loadPlayerStats();
  setPlayerStats(stats);
  if (userId) {
    fetch(`${API_BASE}/api/users/${userId}`)
      .then(res => res.json())
      .then(data => {
        setCoins(data.coins || 0);
        setGems(data.gems || 0);
      })
      .catch(err => console.error('Error loading currency:', err));
  }
}, [userId]);
```

Metodo	Endpoint consumido	Uso en el frontend
POST	/api/auth/login	Login/registro y datos del jugador (App.tsx)
GET	/api/rooms	Listado inicial de salas (App.tsx)
POST	/api/rooms/:id/join	Cambio de equipo / unirse a sala (App.tsx)
PUT	/api/rooms/:id/start	Iniciar partida multijugador (App.tsx)
GET	/api/users/:userId	Perfil, monedas y gemas (LobbyScreen)

5. Componentes principales

Pantalla de acceso en dos pasos (LoginScreen.tsx):

Formulario controlado con validacion de email, envio de codigo de 6 digitos y verificacion antes de llamar a **onLoginSuccess**. Animaciones de entrada/salida con Framer Motion y fondo tematico con particulas:

frontend/src/components/LoginScreen.tsx - verificacion en 2 pasos (extracto real)

```
const [step, setStep] = useState<'email' | 'code'>('email');
const [email, setEmail] = useState('');
const [code, setCode] = useState('');

const handleSendCode = (e: React.FormEvent) => {
  e.preventDefault();
  if (!email || !email.includes('@')) {
    setError('Por favor ingresa un email valido'); return;
  }
  const randomCode = '000000';
  setGeneratedCode(randomCode);
  setStep('code');
};

const handleVerifyCode = (e: React.FormEvent) => {
  e.preventDefault();
  if (code === generatedCode) { onLoginSuccess(email); }
  else { setError('Codigo incorrecto. Intenta de nuevo.'); }
};
```

Lobby con modos de juego (LobbyScreen.tsx):

El lobby muestra perfil, rango con barra de progreso, misiones diarias, top rank y la seleccion de los 5 modos de juego, cada tarjeta dispara **onStartMode(mode)**:

frontend/src/components/LobbyScreen.tsx - tarjetas de modo (extracto real)

```

[[
  {mode: 'classic', icon: Boxes, title: 'CLASICO', desc: 'Encuentra pares'},
  {mode: 'infinite', icon: InfinityIcon, title: 'INFINITO', desc: 'Aguanta tiempo'},
  {mode: 'challenge', icon: Target, title: 'DESAFIO', desc: 'Misiones'},
  {mode: 'boss', icon: Skull, title: 'BOSS', desc: 'Enfrenta IA'},
  {mode: 'ai-friends', icon: Users, title: 'AMIGOS IA', desc: 'Por turnos'}
].map((item) => (
  <div key={item.mode} onClick={() => onStartMode(item.mode)}
    className="mode-card rounded-lg p-3 ..." > ... </div>
)))

```

Sistema de progresion (playerEvolution.ts):

Define los 10 rangos del juego (de Mente Basica a Leyenda Eterna) con umbrales de XP, colores, descripciones y recompensas desbloqueables, consumidos por el lobby, el perfil y el modal "Camino de la Evolucion":

frontend/src/lib/playerEvolution.ts - rangos (extracto real)

```

export interface PlayerRank {
  id: number; name: string; icon: string;
  minXP: number; maxXP: number;
  color: string; gradient: string;
  description: string; rewards: string[];
}

export const RANKS: PlayerRank[] = [
  { id: 1, name: 'Mente Basica', minXP: 0, maxXP: 1000,
    description: 'Iniciando el viaje neural',
    rewards: ['Skin: Clasico', 'Efecto: Basico'] },
  { id: 2, name: 'Sinapsis Activa', minXP: 1000, maxXP: 3000,
    description: 'Las conexiones comienzan a formarse',
    rewards: ['Skin: Electrico', 'Efecto: Chispas', 'Boss: Caos'] },
  // ... hasta 10 rangos (Leyenda Eterna, 180000+ XP)
];

```

6. Capturas de pantalla (interfaces renderizadas)

Capturas tomadas el 5 de octubre de 2026 sobre el frontend real corriendo en **http://localhost:5173** (servidor de desarrollo Vite, build del código del repositorio), navegador Chrome 1366x768, flujo completo: acceso, lobby, modos de juego, tienda, ranking y perfil.

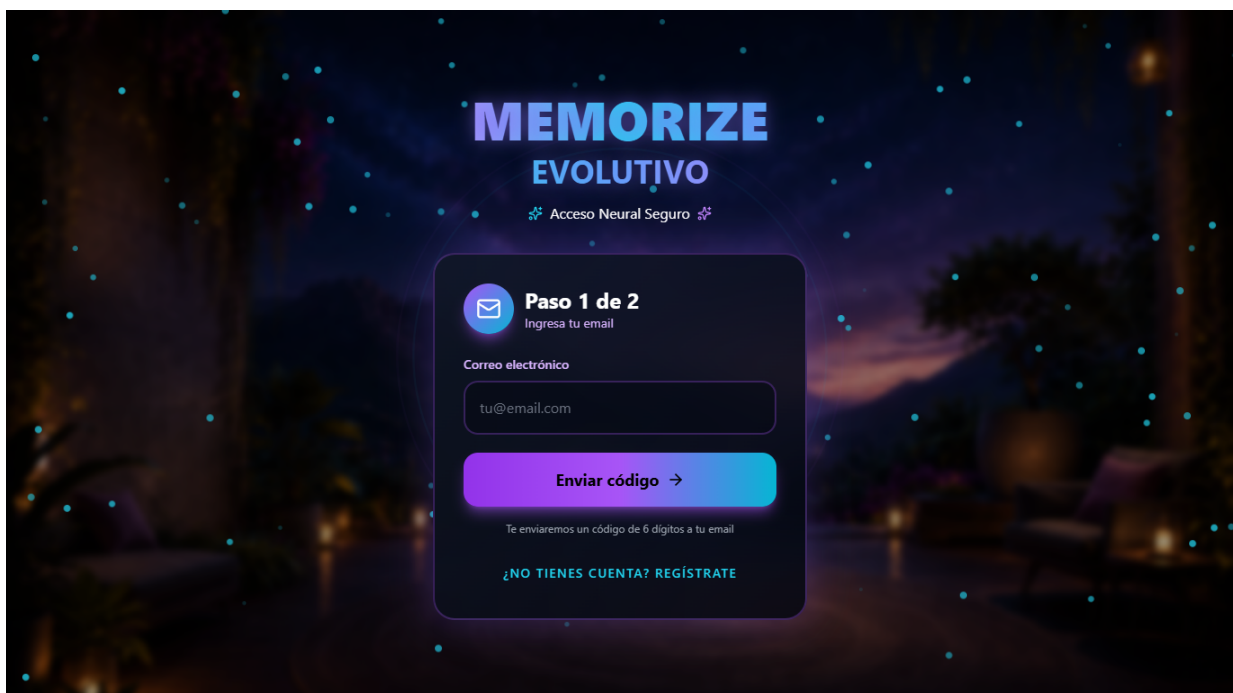


Figura 1 - Pantalla de acceso (LoginScreen, paso 1 de 2).

Acceso neural con email y código de verificación de 6 dígitos, animaciones Framer Motion, partículas y fondo temático. Enlace a registro en la parte inferior.

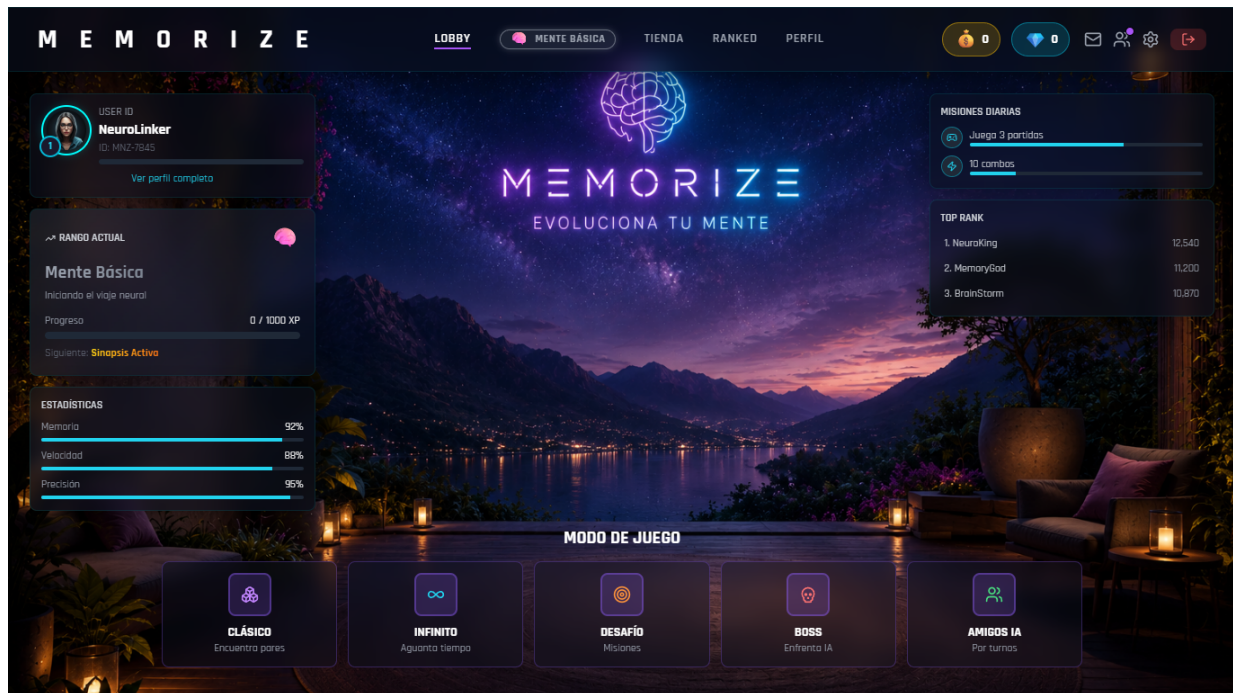


Figura 2 - Lobby principal tras el login.

Cabecera con navegación (Lobby, Tienda, Ranked, Perfil), monedas/gemas del usuario servidas por la API, tarjeta de perfil con rango "Mente Basica" y progreso XP, misiones diarias, top rank y selector de los 5 modos de juego. Fiel al prototipo de menu principal.

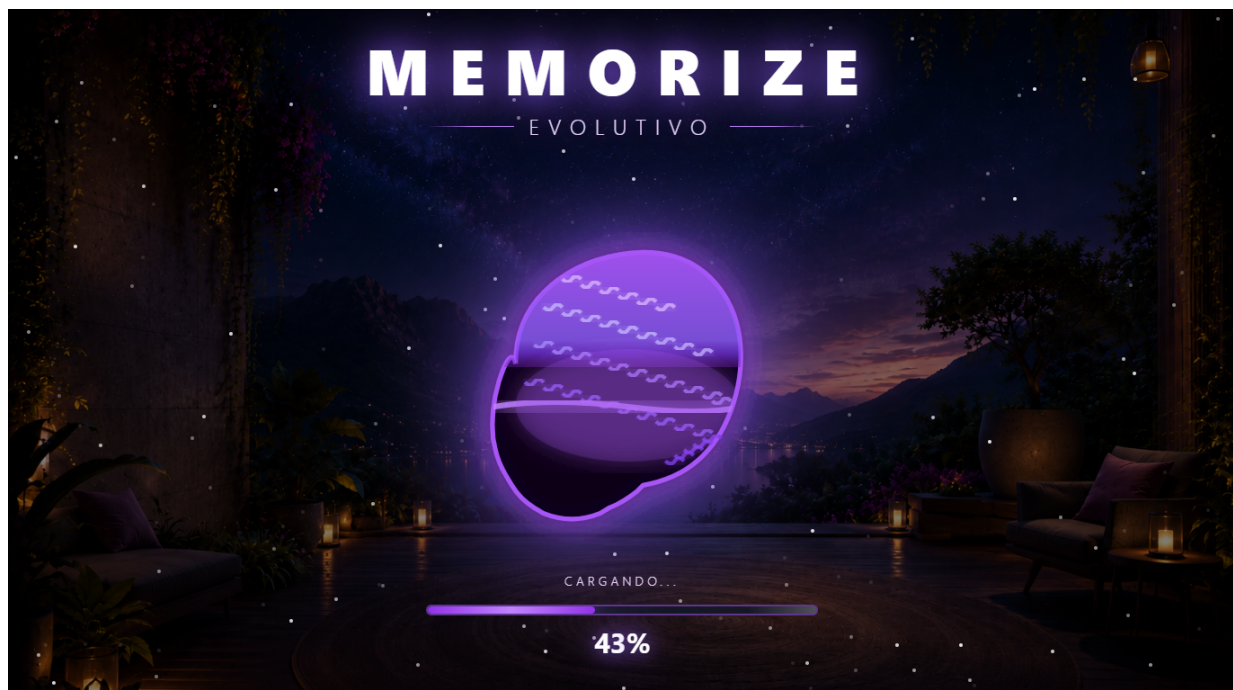


Figura 3 - Pantalla de carga entre secciones (LoadingScreen).

Transición animada con cerebro que se llena según el progreso (0-100 %) y barra de carga, usada antes de cada modo para una experiencia fluida.

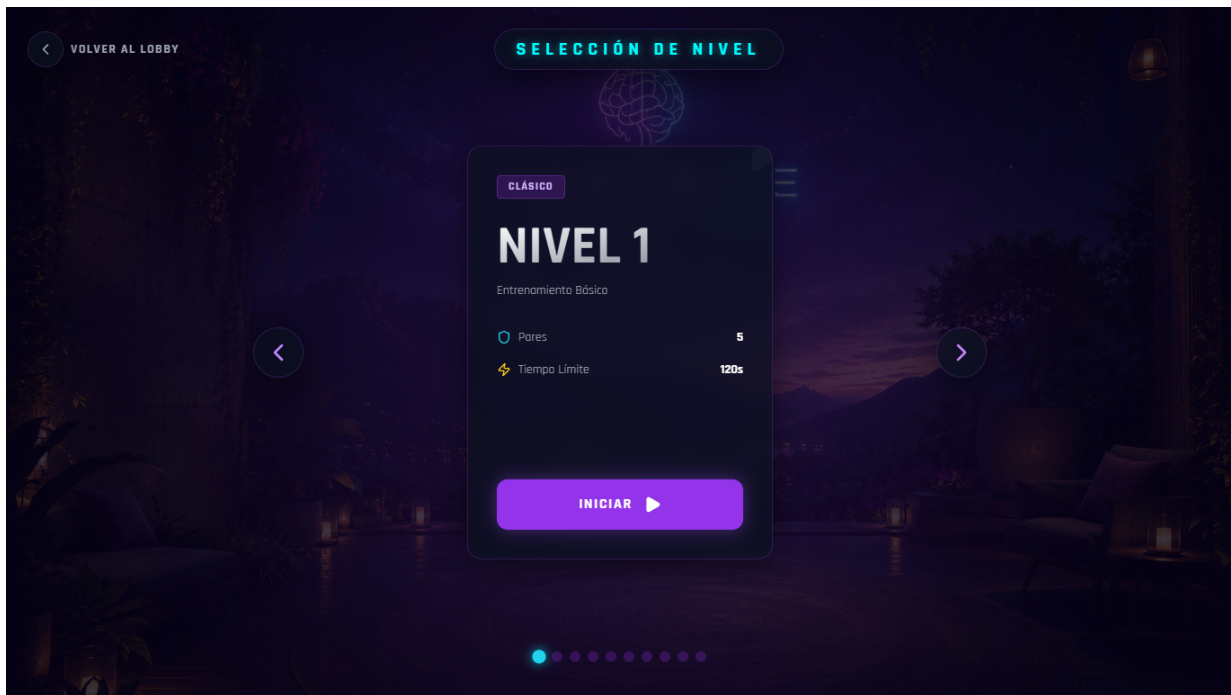


Figura 4 - Selector de niveles del modo Clasico.

10 niveles progresivos de dificultad creciente (de 4 a 24 cartas), con niveles bloqueados segun avance, tal como define el SRS del modo Clasico.

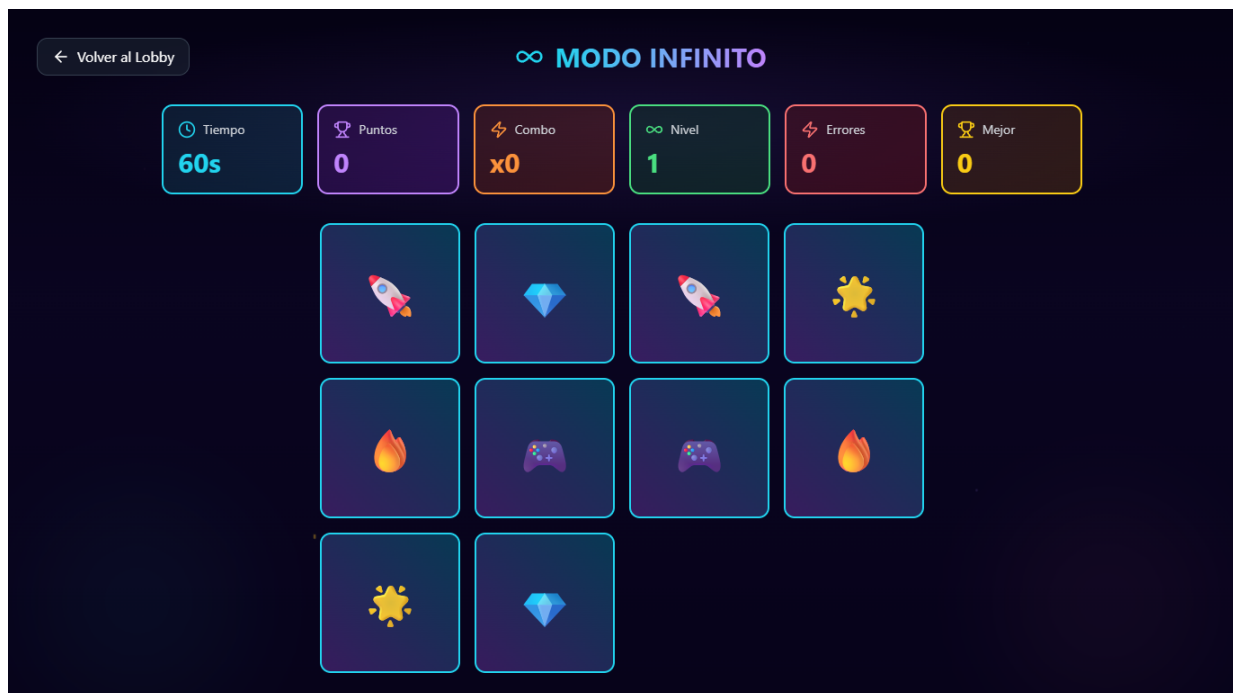


Figura 5 - Partida del modo Infinito en curso.

Tablero de memoria funcional con HUD en vivo: tiempo, puntos, combo, nivel, errores y mejor marca. Las cartas se voltean y emparejan con sonidos y efectos.

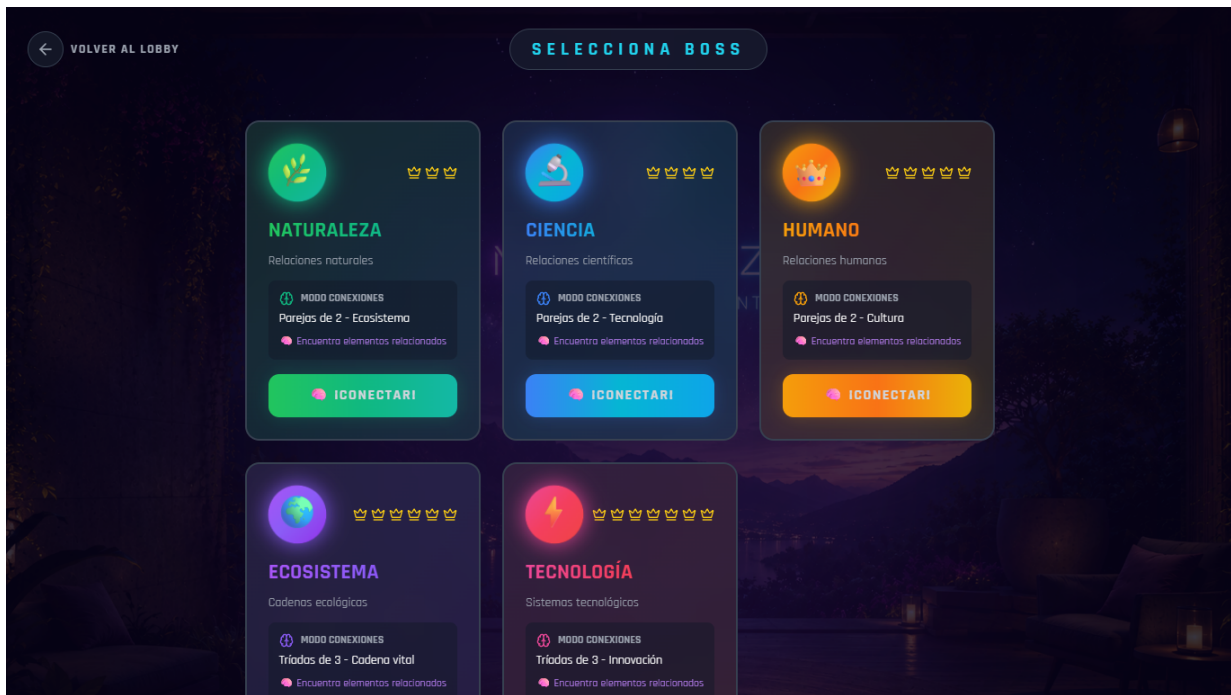


Figura 6 - Selector de jefes (Boss Fight).

Cinco jefes con mecanicas unicas, desbloqueables por rango del jugador, previo al combate.



Figura 7 - Tienda de personalizacion.

Packs de cartas (Frutas, Animales, Espacial, Oceano...), marcos, skins y tableros comprables con monedas/gemas, con estados Desbloqueado/Equipado sincronizados con el inventario.

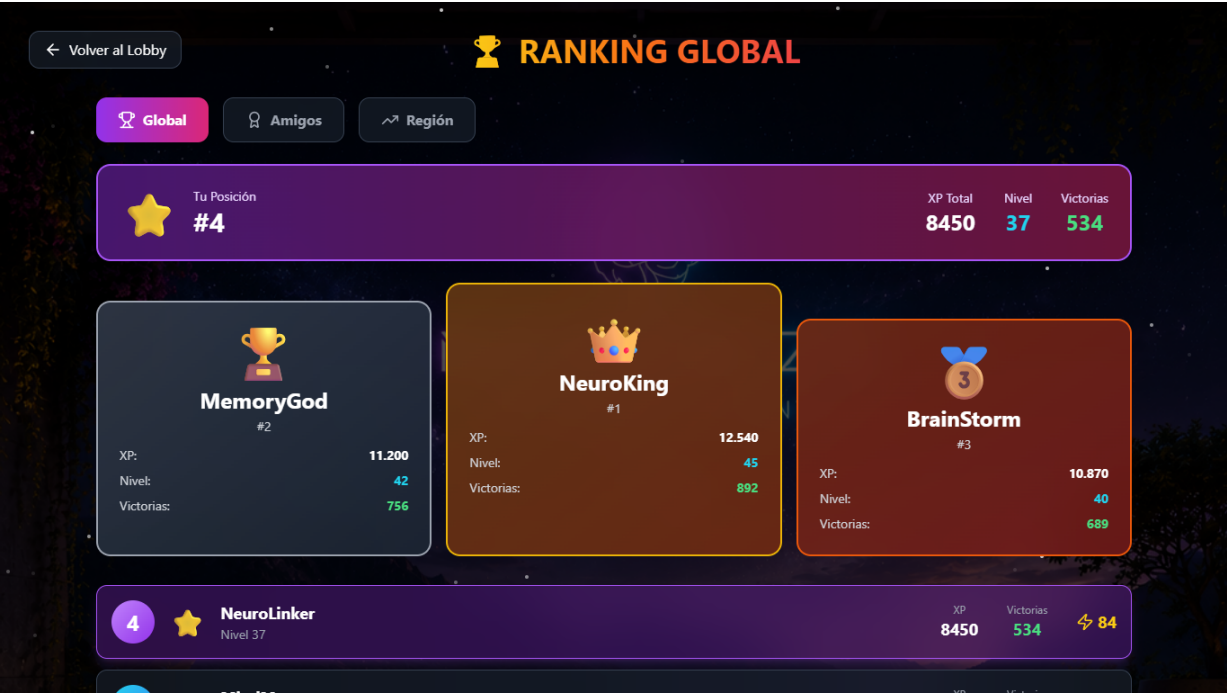


Figura 8 - Ranking global.

Tabla de posiciones con podio y lista completa, base de los rankings global, por modo, semanal y mensual que expone el backend.



Figura 9 - Perfil del jugador.

Nivel, XP, rango, estadísticas (partidas, victorias, mejor puntaje, combos) y progreso, alimentados por el sistema de evolucion y la API de usuarios.

7. Conclusion y cumplimiento

Criterio de la entrega	Estado
Repositorio publico con el codigo del frontend	Cumple: https://github.com/kevinmora18/memorize (carpeta frontend/, rama main)

Fragmentos de código del componente principal y conexión API	Cumple: App.tsx, socket.ts, LoginScreen, LobbyScreen, playerEvolution (secciones 3-5)
Capturas renderizadas en navegador, fieles al prototipo	Cumple: 9 figuras (login, lobby, carga, niveles, infinito, boss, tienda, ranking, perfil)
Interfaz responsiva	Cumple: Tailwind CSS con diseños adaptativos por pantalla
Comunicación con el servidor	Cumple: API REST (fetch) + WebSocket (Socket.IO) con respaldo ante fallos

El frontend de Memorize Evolutivo está implementado al 100 % según el SRS: más de 35 componentes React funcionales, 6 modos de juego, sistema de progresión de 10 rangos, tienda completa, panel de administración y comunicación en tiempo real con el backend. Las capturas demuestran interfaces renderizadas reales, fieles al prototipo y al diseño neonal del juego.

Fin de la Evidencia 2 - 29 de septiembre de 2026